

CernVM-FS Filebundles for low-latency software distribution in interactive use cases

Valentin Volk^{1,*}, Georgios Christodoulis¹, and Jakob Blomer¹

¹CERN, 1211 Geneva 23, Switzerland

Abstract. CernVM-FS is a read-only, on-demand file system used to distribute scientific software to the Worldwide LHC Computing Grid and many communities beyond high-energy physics. Its lazy, per-file fetching is highly efficient for batch computing, but it adds noticeable latency to interactive sessions: starting an application that opens hundreds of small files on a cold cache serializes one network round trip after another. We present *filebundles*, a mechanism that exploits the predictable file-access pattern at application startup. A small bundle specification, published next to a trigger file, declares the set of files needed immediately after the trigger is opened. When a client opens the trigger, CernVM-FS prefetches the listed dependencies concurrently using a pool of worker threads, replacing a long serial chain of requests with a short parallel prefetch phase. On a ROOT startup benchmark, filebundles bring performance into the local-disk range and keep startup time nearly flat as network latency grows, while the on-demand baseline degrades approximately linearly. Filebundles are an opt-in feature in CernVM-FS 2.14.

1 Introduction

CernVM-FS [1] is a global, read-only file system for distributing scientific software. Rather than downloading and unpacking a complete software stack in advance, clients access a global namespace under `/cvmfs`. Files are fetched when they are opened; downloading, decompression, and caching are transparent to the user.

CernVM-FS is a core component of the Worldwide LHC Computing Grid and is also used by communities beyond high-energy physics, including Euclid, LIGO, LSST, SKA, and EESSI. The infrastructure has about fifteen Stratum 1 mirror servers in Europe, the Americas, and Asia. The `/cvmfs` namespace contains about 3.3×10^{10} files and 2 PB of data; it also serves more than 8000 container images.

This lazy, content-addressed model is extremely effective for distributed batch computing, where the benefits of a shared software distribution system dominate any per-file overhead. In interactive sessions, however, the same on-demand behaviour can make application startup noticeably slower than from local disk. This paper presents *filebundles*, a mechanism that reduces that startup latency by prefetching the files an application needs immediately after launch.

*e-mail: valentin.volk@cern.ch

2 Motivation

At the CernVM Users Workshop in 2022, Chris Burr (LHCb) reported that interactive CernVM-FS use can be roughly three to five times slower than local disk, even with a hot cache [2]. This overhead is usually acceptable for distributed batch workloads, but is noticeable when starting an interactive analysis environment.

The effect is easy to reproduce on a controlled testbed. Installing ROOT from a release tarball on local disk, importing ROOT in Python takes about 2.47 s. Using the same software stack from `/cvmfs` with a cold cache, the time increases to about 4.69 s. In this setup the network path is very short, so the slowdown is not primarily a bandwidth problem. It confirms that a significant part of the delay comes from the way CernVM-FS serves many files on demand.

2.1 Where does the time go?

When an application such as ROOT starts, it opens many files one after another. On a cold cache, for each file CernVM-FS handles a FUSE upcall, issues an HTTP request, waits for the round trip, downloads the object, decompresses it and returns the data. This per-file overhead is paid again and again, and because the requests are dependent they are effectively serialized. The total amount of work is unremarkable, but the serialized per-file latency adds up. In batch computing this trade-off is acceptable; in interactive use the serialization is precisely what hurts.

It is useful to recall the CernVM-FS cache hierarchy (Fig. 1) in which any optimization has to operate. At the top is the Stratum 0, the writable origin managed by the repository maintainer (“librarian”). Below it are the globally distributed Stratum 1 read-only mirrors. Sites typically add a Squid forward proxy to absorb repeated requests from many clients. On each worker node or workstation CernVM-FS keeps a local disk cache, and above that the Linux kernel maintains the page cache. Filebundles operate within this existing hierarchy. They do not change the data model; they only change *when* and *how* the first wave of file requests is issued.

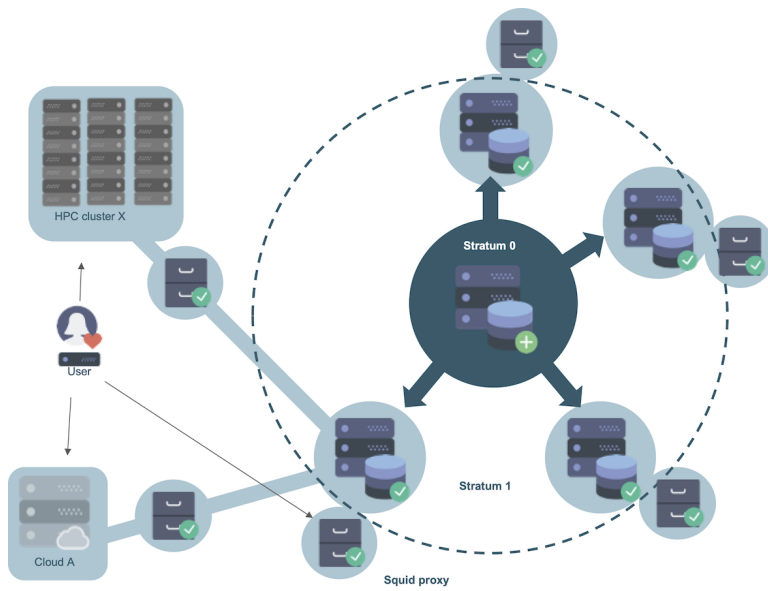


Figure 1. The CernVM-FS cache hierarchy: a writable Stratum 0 origin, globally distributed Stratum 1 mirrors, site-level Squid proxies, and per-client disk and kernel page caches. Filebundles change only the timing of the first wave of requests, not the data model.

3 Filebundles

Filebundles use prior knowledge of application startup. For applications such as ROOT [3] or Python, the files needed immediately after a given *trigger* file is opened are often predictable. A bundle records these dependencies instead of leaving the client to discover them one file at a time.

A bundle is declared by a specification file published next to its trigger file in the same directory, named `.cvmfsbundle-<filename>`, where `<filename>` is the trigger. The specification is a small, versioned JSON document holding minimal metadata and a flat list of dependency paths, expressed as absolute paths from the repository root. A representative ROOT bundle, keyed on the Python package `__init__.py`, contains roughly 250 paths:

```
##CVMFS_BUNDLE version=1 encoding=UTF-8
{
  "name": "CVMFS_BUNDLE",
  "version": "1.0.0",
  "encoding": "UTF-8",
  "dependencies": [
    "/root/lib/libCore.so",
    "/root/lib/libRIO.so",
    "/root/lib/libcppy.so",
    "/root/lib/ROOT/_application.py",
    ... (~250 paths total)
  ]
}
```

When a client opens the trigger file, CernVM-FS detects the matching `.cvmfsbundle` specification and begins prefetching the listed dependencies concurrently, using a pool of worker threads, while the application continues to run. In effect, a long serial chain of dependent requests is replaced by a much shorter parallel prefetch phase that warms the local cache ahead of the application's subsequent `open()` calls.

The feature is disabled by default and can be enabled per mount with `CVMFS_PREFETCH_FILEBUNDLES=yes` [4]. The setting is read at mount time; mounts for which it is disabled do not start bundle prefetching. The prefetch-pool size is controlled by `CVMFS_BUNDLE_POOL_SIZE`.

3.1 Publishing a filebundle

From the librarian's point of view the workflow is straightforward. First, choose a suitable trigger file, for example a Python `__init__.py` or a launcher binary such as `root.exe`. Second, determine which files are typically read after that trigger; this can be done by hand or with tracing tools such as `strace` or the built-in `CVMFS_TRACEFILE` mechanism. Third, publish the resulting `.cvmfsbundle-<filename>` specification next to the trigger file, just like any other file in the repository. Removing the specification (and republishing) removes the bundle, after which the trigger behaves like an ordinary file.

4 Results

Unless noted otherwise, the benchmarks use ROOT 6.36.12 in a test CernVM-FS repository served from `localhost`. The client ran on an AMD EPYC 7302 system (2×16 cores, 64 threads at ~3.0 GHz, 251 GB RAM), with AlmaLinux 10.1 and kernel 6.12. Using `localhost` removes external network effects, exposing the per-file software overhead.

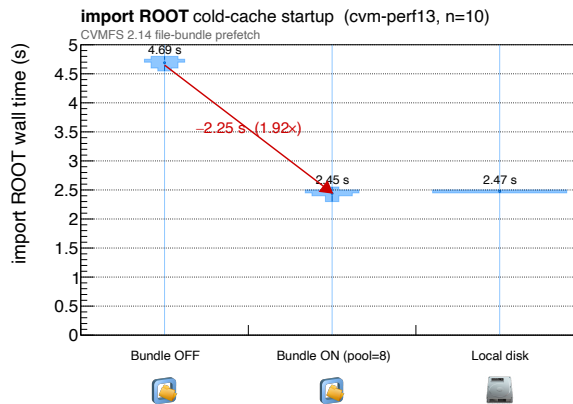


Figure 2. Distribution of ROOT startup times. With filebundles enabled (CVMFS_PREFETCH_FILEBUNDLES=ON), CernVM-FS startup matches local-disk performance, closing the gap to the on-demand cold-cache baseline.

4.1 Startup performance

Figure 2 compares ROOT startup times on local disk, on CernVM-FS with a cold cache and no filebundles, and on CernVM-FS with filebundles enabled. In this benchmark, filebundles bring CernVM-FS startup time to the local-disk range. Some measurements are marginally faster than local disk, consistent with overlap between downloading and decompression in the prefetch path.

4.2 Prefetch thread pool

Prefetching is implemented with a pool of worker threads whose size is set by CVMFS_BUNDLE_POOL_SIZE. As shown in Fig. 3, startup performance improves as workers are added but saturates around eight threads, which is also the default. Most of the benefit is therefore obtained with a moderate amount of concurrency and without aggressive tuning.

4.3 Network latency

We injected loopback round-trip latency with `tc qdisc netem`. Without filebundles, startup time grows approximately linearly with round-trip time because the application waits for many dependent, small-file fetches. With filebundles, startup time remains nearly flat over a substantial range (Fig. 4): concurrent prefetch pays the round-trip penalty for a small number of batches rather than for each file. This is an idealized loopback experiment.

4.4 Caveats

Filebundles do not reduce the total amount of data transferred; they compress the startup traffic into a short, intense burst. In the benchmark, requests peak at about 1480 per second with filebundles, versus about 680 per second without them (Fig. 5). The feature should therefore be used selectively, in particular on interactive systems such as `lxplus`, where startup latency matters most. Over-prefetching should be avoided, and any rollout should be coordinated with proxy and site administrators so that the request bursts do not overload shared caches.

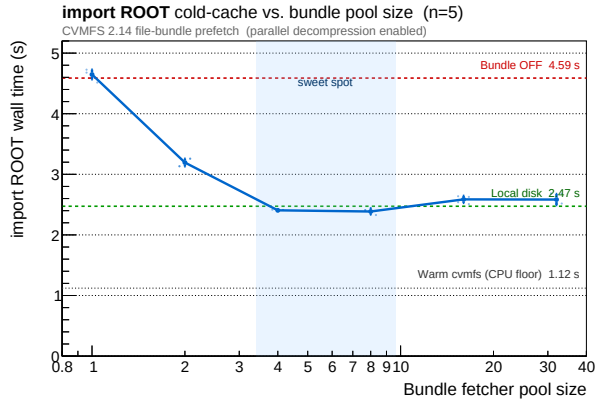


Figure 3. ROOT startup time as a function of the prefetch thread-pool size (CVMFS_BUNDLE_POOL_SIZE). Performance saturates around eight worker threads, the default.

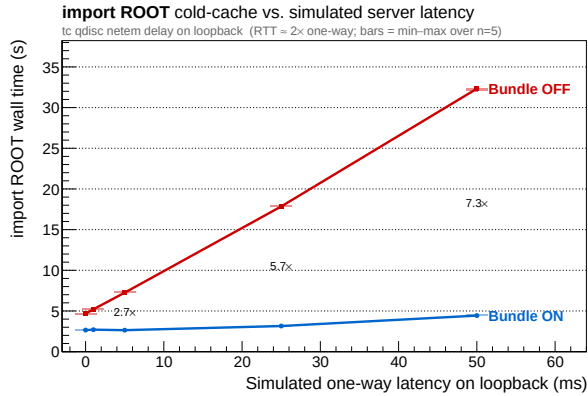


Figure 4. ROOT startup time as a function of injected round-trip latency. The on-demand baseline degrades roughly linearly, while filebundles keep startup nearly flat by overlapping requests.

5 Outlook

FUSE passthrough in CernVM-FS 2.14 on recent kernels can hand the kernel a descriptor for the cached backing file at `open()`, so subsequent reads bypass FUSE. Its interaction with filebundles still needs to be benchmarked. Planned work for CernVM-FS 2.15 includes tools for creating and updating bundles, support for file chunks in bundle specifications, and integration with build tools. Packfiles, which would combine small backend objects to reduce request counts, are also under consideration; backward compatibility is the principal obstacle.

6 Conclusions

Filebundles reduce application-startup latency when a known set of files is needed immediately after a trigger is opened. They require the repository maintainer to trace startup behaviour and publish a `.cvmfsbundle` specification next to the trigger. In the ROOT benchmark, they reduce cold-start time to the local-disk range and make startup less sensitive to

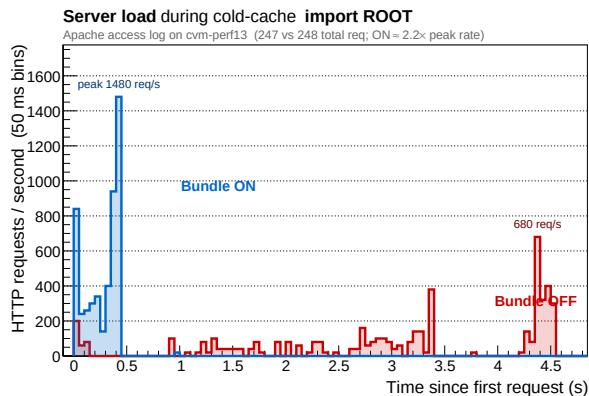


Figure 5. Request rate during startup. Filebundles concentrate the same total traffic into a shorter, more intense burst (peak ~ 1480 req/s versus ~ 680 req/s without bundles).

added round-trip latency. Filebundles are an opt-in CernVM-FS 2.14 feature, enabled per mount with `CVMFS_PREFETCH_FILEBUNDLES`. They are best suited to interactive systems, where the prefetch burst can be managed.

Acknowledgements. We thank Chris Burr (LHCb) for the input that motivated this work, and the CernVM-FS team at CERN.

Disclosure: AI-based tools were used to assist with drafting and formatting this manuscript; all content was reviewed and verified by the authors.

References

- [1] J. Blomer, C. Aguado-Sánchez, P. Buncic, A. Harutyunyan, Distributing LHC application software and conditions databases using the CernVM file system, *J. Phys.: Conf. Ser.* **331**, 042003 (2011). <https://doi.org/10.1088/1742-6596/331/4/042003>
- [2] C. Burr, *Client performance*, CernVM Users Workshop (2022). https://indico.cern.ch/event/1079490/contributions/4939521/attachments/2506388/4306645/2022-09-12_CVMFS-Workshop_LHCb.pdf
- [3] R. Brun, F. Rademakers, ROOT—an object oriented data analysis framework, *Nucl. Instrum. Methods Phys. Res. A* **389**, 81–86 (1997). [https://doi.org/10.1016/S0168-9002\(97\)00048-X](https://doi.org/10.1016/S0168-9002(97)00048-X)
- [4] CernVM-FS documentation. <https://cvmfs.readthedocs.io/>